

WEEK 1: CUSTOMER MANAGEMENT SYSTEM

Introduction

The Customer Management System is designed to store and manage customer information for an e-commerce application. The system maintains important customer details such as Customer ID, First Name, Last Name, Email, Phone Number, and Created Date.

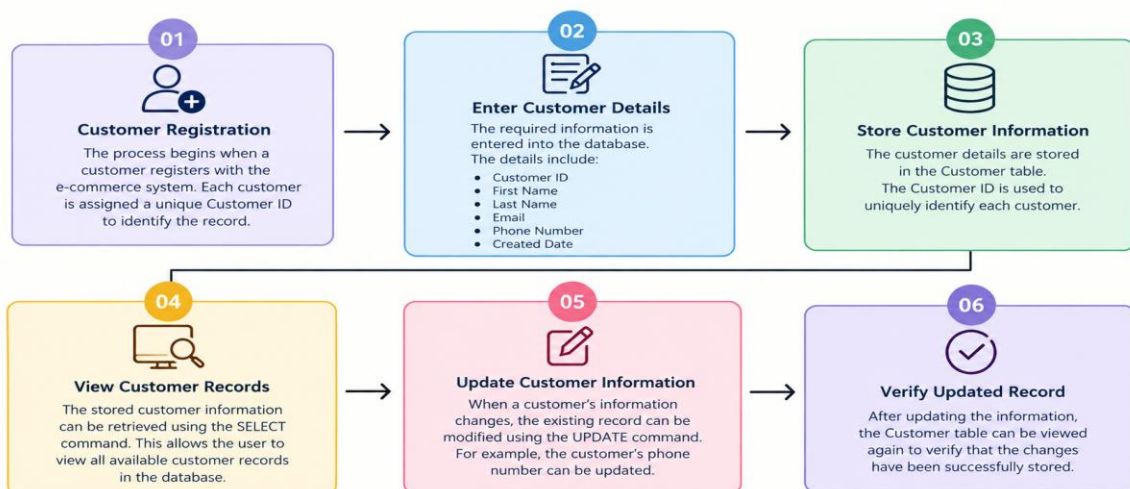
SQL commands are used to create the Customer table, insert customer records, retrieve stored information, and update existing customer details. This helps maintain accurate and organized customer information in the database.

Objective

The main objectives of the Customer Management System are:

- To create a structured Customer table.
- To store customer information in the database.
- To uniquely identify each customer using Customer ID.
- To retrieve and display customer records.
- To update customer information when required.
- To maintain accurate and consistent customer data.
- To understand basic SQL operations such as CREATE, INSERT, SELECT, and UPDATE.

The Process Flow



The Customer Management System follows a systematic process for managing customer information.

Customer Registration → Enter Customer Details → Store Customer Information → View Customer Records → Update Customer Information → Verify Updated Record

01. Customer Registration

The process begins when a customer registers with the e-commerce system. Each customer is assigned a unique Customer ID to identify the customer record.

02. Enter Customer Details

The required information is entered into the database. The details include:

- Customer ID
- First Name
- Last Name
- Email
- Phone Number
- Created Date

03. Store Customer Information

The customer details are stored in the **Customer** table. The Customer ID is used to uniquely identify each customer.

04. View Customer Records

The stored customer information can be retrieved using the SELECT command. This allows the user to view all available customer records in the database.

05. Update Customer Information

When a customer's information changes, the existing record can be modified using the UPDATE command. For example, the customer's phone number can be updated.

06. Verify Updated Record

After updating the information, the Customer table can be viewed again to verify that the changes have been successfully stored.

Customer Table Structure

Attribute	Data Type	Description
Customer_ID	VARCHAR(20)	Unique identification number of the customer
First_Name	VARCHAR(20)	Customer's first name
Last_Name	VARCHAR(20)	Customer's last name
Email	VARCHAR(30)	Customer's email address
Phone	NUMBER(10)	Customer's phone number
Created_At	DATE	Date on which the customer record was created

Database Operations Performed

Creating the Customer Table

The Customer table was created to store customer-related information. The table contains fields for customer identification, name, email, phone number, and registration date.

```
SQL> DESC customer
Name                               Null?    Type
-----
CUSTOMER_ID                       NOT NULL VARCHAR2(20)
FIRST_NAME                        VARCHAR2(20)
LAST_NAME                         VARCHAR2(20)
EMAIL                             VARCHAR2(30)
PHONE                             NUMBER(10)
CREATED_AT                        DATE
```

Inserting Customer Records

Customer records were inserted into the Customer table using the INSERT INTO command.

The project includes customer records such as:

- C-1001 – Ricky Ponting
- C-1002 – Calvin Grey

```
SQL> INSERT INTO customer (customer_id, first_name, last_name, email, phone,
created_at)
2 VALUES (
3     '&cust_id',
4     '&first_name',
5     '&last_name',
6     '&email',
7     &phone_num,
8     DATE '&created_date'
9 );
Enter value for cust_id: C-1002
old 3:      '&cust_id',
new 3:      'C-1002',
Enter value for first_name: Calvin
old 4:      '&first_name',
new 4:      'Calvin',
Enter value for last_name: Grey
old 5:      '&last_name',
new 5:      'Grey',
Enter value for email: calvingrey@edss.com
old 6:      '&email',
new 6:      'calvingrey@edss.com',
Enter value for phone_num: 8765432109
old 7:      &phone_num,
new 7:      8765432109,
Enter value for created_date: 2024-09-23
old 8:      DATE '&created_date'
new 8:      DATE '2024-09-23'

1 row created.
```

Viewing Customer Records

The SELECT command was used to retrieve the customer information stored in the database.

```
SQL> select * from customer
```

- This displays the available Customer ID, First Name, Last Name, Email, Phone Number, and Created Date.
- The retrieved records can be used to verify whether the customer information has been correctly stored.

Updating Customer Information

- The UPDATE command was used to modify existing customer information.
- For example, the phone number of customer **C-1002** was updated.

```
SQL> UPDATE customer SET phone = 9870654312 WHERE customer_id = 'C-1002';  
  
1 row updated.
```

Verification

After performing the insert and update operations, the Customer table was displayed using the SELECT command.

```
SQL> select * from customer
```

The output was checked to verify that:

- Customer records were successfully inserted.
- Customer IDs were stored correctly.
- Customer names and email addresses were stored.
- Phone numbers were updated correctly.
- Created dates were stored in the required format.

Result

The Customer Management System was successfully implemented using SQL. Customer details were created, inserted, displayed, and updated successfully.

The project demonstrates the use of basic SQL database operations for managing customer information in an e-commerce system.

Conclusion

The Week 1 Customer Management System provides a simple and organized way to manage customer information. The Customer table stores essential customer details, while SQL commands allow the data to be inserted, retrieved, and updated.

Through this task, the basic concepts of **table creation, data insertion, data retrieval, and data modification** were implemented successfully. The system can also be extended in future weeks by connecting customer information with other e-commerce modules such as products, orders, sellers, and inventory.